



ACCELERATING
DIGITAL
EVOLUTION

DevOps Solution



TRIANZ CONFIDENTIAL





ACCELERATING
DIGITAL
EVOLUTION

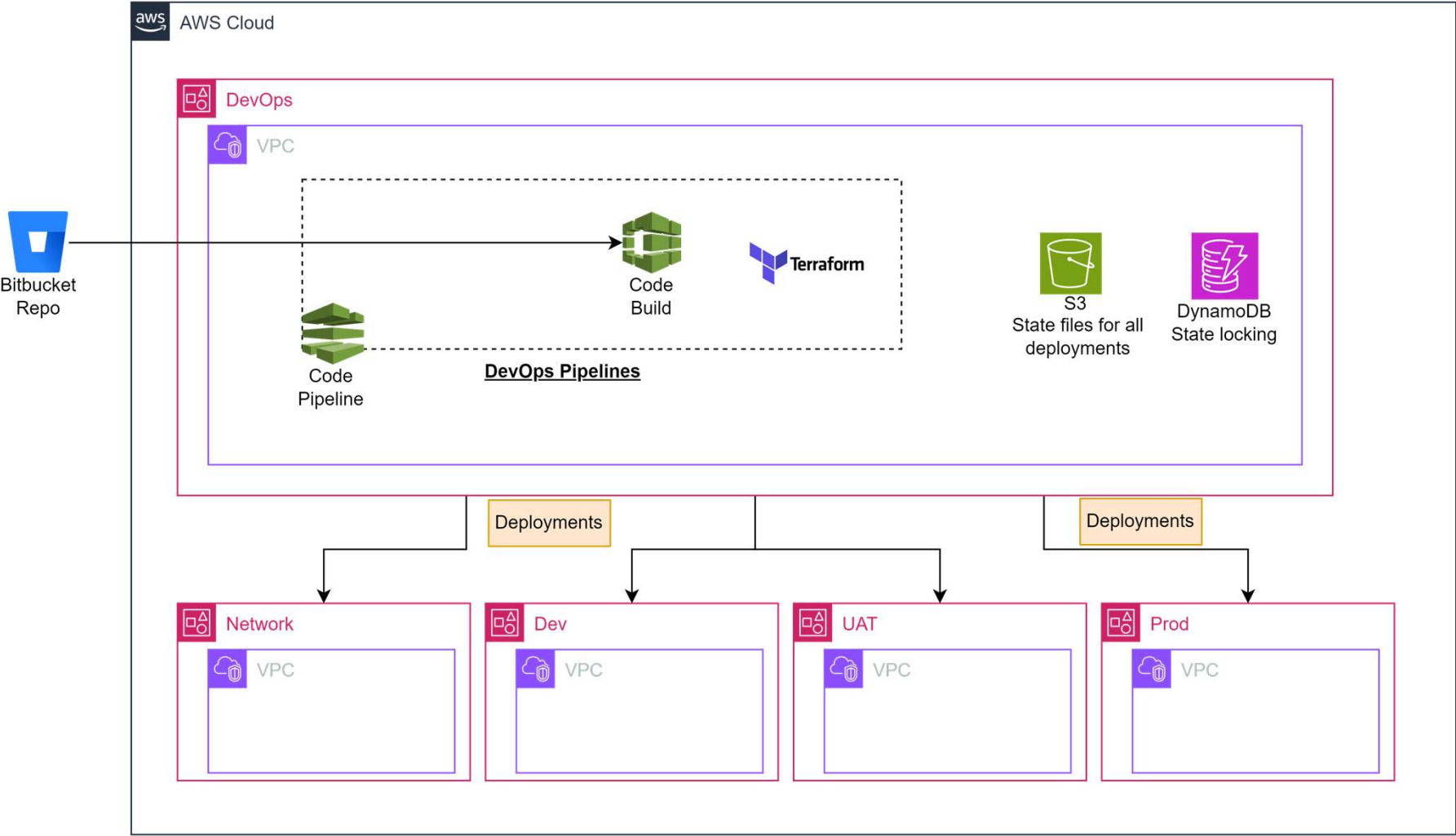
CICD architecture



Overall CI/CD Solution



- AWS environment is segregated and secured into multi account structure as shown in the diagram
- Deployment into all environment is done through DevOps pipelines in the DevOps account
- Bitbucket is used as code repository and Terraform is the IaC (preferred) for all deployments
- The state of your managed infrastructure and configuration is recorded in a state file which is stored in S3 bucket and locking is achieved by writing to a DynamoDB table



Deployment Design



All workloads are deployed in three environments

- **Dev**
 - App-dev account will be used for this purpose
 - All the development activities (POCs, initial development) will be done in this environment.
 - All developers will have access to this environment
- **UAT**
 - App-UAT account will be used for this purpose
 - All the internal testing (by Trianz) and Customer UAT will be done in this environment
 - No change will be allowed by the developers in this environment
- **Prod**
 - App-Prod account will be used for this purpose
 - All Live / Production workloads will be deployed in this account
 - No change will be allowed by developers in this environment

laC, Repository and Branches

- Bitbucket will be used as the code repository
- Three branches (dev, uat and main-prod) corresponding to each environment will be created

CICD Pipelines:

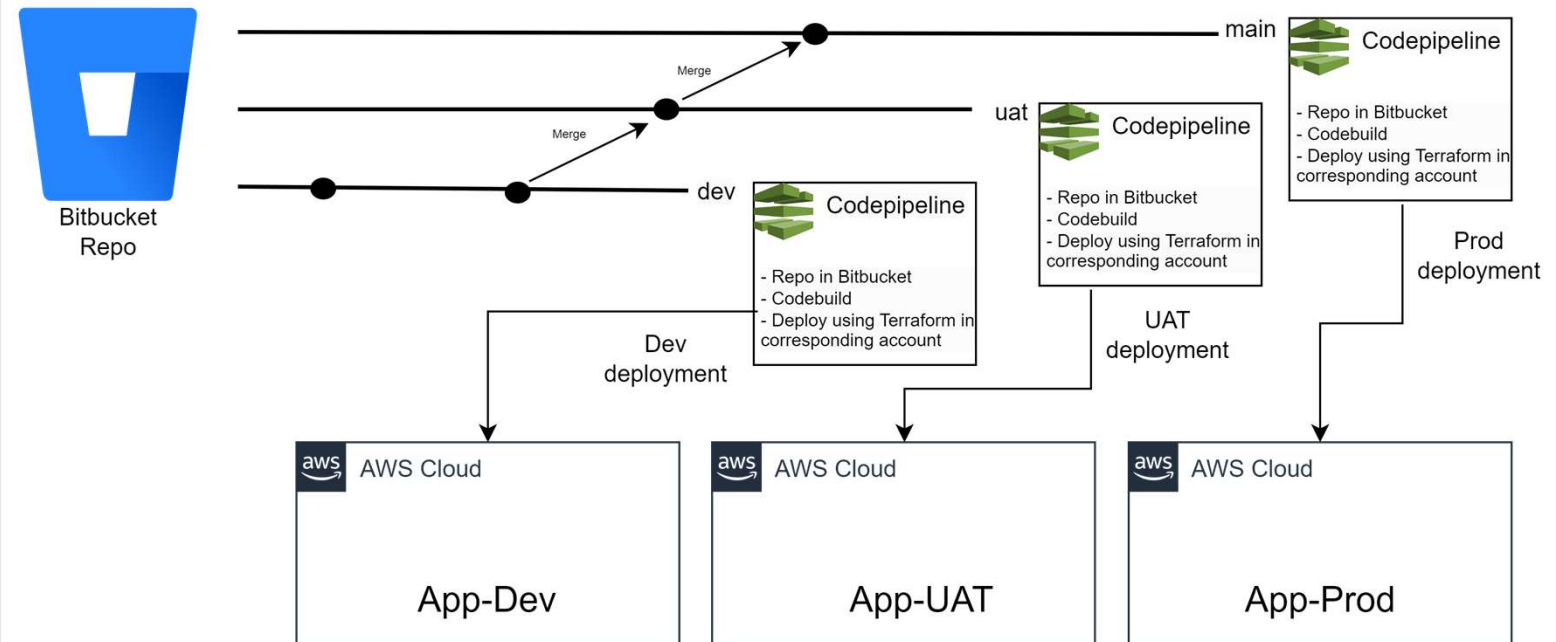
- One code pipeline will be mapped to each corresponding branch and deployment will be triggered to corresponding AWS account.
- Each repo in Bitbucket will have similar pipeline in DevOps account as depicted in diagram

Repositories:

- Please refer to the excel file attached.

Code promotion:

- Anyone can merge the code in the Dev environment for every user story.
- Promotion from Dev to UAT stage will need approval from Architect or track lead
- Promotion from UAT to Prod stage will need architect approval. Post the go live, customer and project manager will also be added to the approval list.

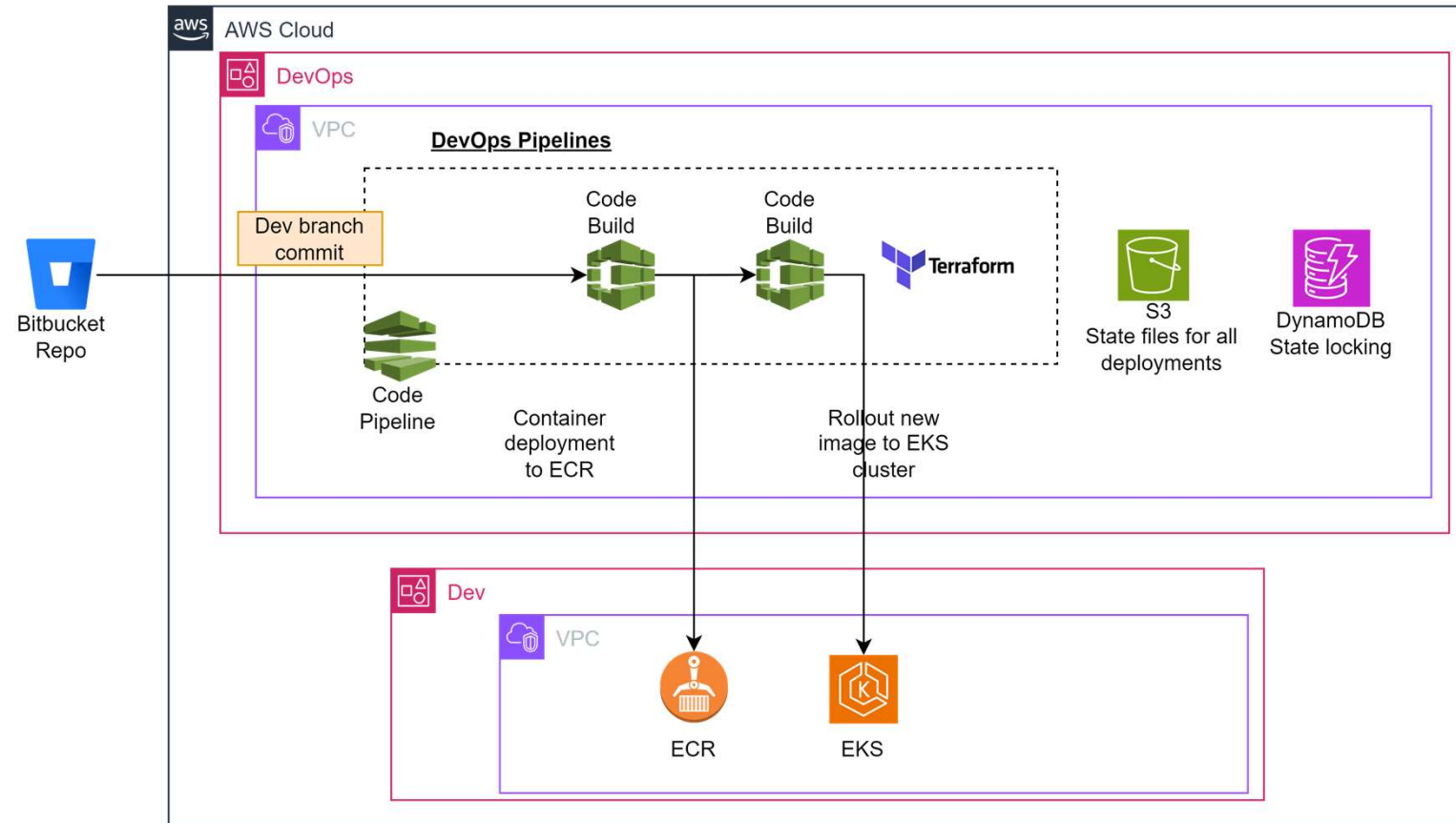


CICD for Application Code rollout



CICD for Application (container) code:

- While the CICD design for the Application code rollout is same as the infrastructure for the most part, the deployment will differ as the code (Java) is first containerized into a docker image, stored ECR and then new image rollout to EKS
- Bitbucket is utilized as the code repository for the application code (container).
- Each repository must have three branches (Dev, UAT and Prod), like infrastructure. There must be three pipelines one for each branch. A single branch and pipeline design is depicted in the diagram attached.
- A new code push on a specific branch will trigger CodePipeline, initiating the CICD flow.
- Once the code is built, a container image is built based on the Dockerfile and the application source code.
- As the last step, Codebuild triggers the newly created container image to EKS as a deployment using the Helm Kubernetes package manager.



CICD tech stack



Stack Name	Vendor Name	Service Definition	Purpose
Bitbucket	Atlassian	Secured, fully managed, highly scalable source control service that hosts private Git repositories.	Storing the source code for all the workloads
CodePipeline	AWS	Fully managed continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates.	Building pipelines triggering the deployment to AWS environment with different stages (approval, build, deployment, testing)
Terraform (preferred for IaC)	Hashicorp	Open-source infrastructure as code (IaC) tool that enables you to define and provision a datacenter infrastructure using a high-level configuration language	Terraform is used as preferred IaC for this project.
CloudFormation (used for Landing zone)	AWS	AWS CloudFormation lets you model, provision, and manage AWS and third-party resources by treating infrastructure as code.	Bootstrap account to tightly couple it with payer account. Also deploys infrastructure via pipeline.
CDK (used for Landing zone)	AWS	AWS Cloud Development Kit (AWS CDK) accelerates cloud development using common programming languages to model your applications.	Deploys AWS resources via application code.

Repositories list



Owner	Repo purpose	Detail	Repo Name	Where
DevOps	Infrastructure-Base	VPC, Subnets	infra-base	Bitbucket
DevOps	Infrastructure - API Infra	EKS, Cognito, ECR, API, Route53,	infra-microservices	Bitbucket
DevOps	Frontend-Infra	S3 Bucket, CloudFront, ACM	frontend-infra	Bitbucket
DevOps	HELM Charts	Helm charts	helm-charts	Bitbucket
DevOps	Pipelines	CodeCommit, Codebuild, Code pipelines	devops-pipelines	Bitbucket
Portal	Front end code	Angular code	frontend-react	Bitbucket
Portal	Customers API.. More such			
	API Microservices	Code for container	NOT FINALISED	Bitbucket
Fleet mgmt	Fleet -Infra	IOT Core, IOT Fleetwise,	fleet-infra	Bitbucket
Contact centre	Contact centre code	All contact center related services	contact-center	Bitbucket
DevOps	All datastores - RDS, DynamoDB, etc	RDS, DynamoDB	data-stores	Bitbucket
Datalake	All data related infra	All data services - Glue, DMS, Appflow,	data-lake	Bitbucket
DevOps	master-pipelines	Code for creating all the pipelines	master-pipelines	Bitbucket

Folder structure for Terraform code and state files



Terraform code folder structure

```
infra-base/
├── buildsspecs/
│   ├── buildspec_apply.yml
│   └── buildspec_plan.yml
├── environments/
│   ├── dev/
│   │   ├── backend.tf
│   │   └── terraform.tfvars
│   └── uat/
│       ├── backend.tf
│       └── terraform.tfvars
├── .gitignore
├── main.tf
├── output.tf
├── provider.tf
├── README.md
└── variables.tf
```

S3 Folder structure for state information

Pipeline	Path for statefile
master-pipeline	- /pipeline/terraformstate.tf
dev	- /dev/base-infra/terraformstate.tf - /dev/infra-microservices/terraformstate.tf - /dev/frontend-infra/terraformstate.tf - /dev/contact-center/terraformstate.tf ..
uat	- /uat/base-infra/terraformstate.tf - /uat/infra-microservices/terraformstate.tf - /uat/frontend-infra/terraformstate.tf - /uat/contact-center/terraformstate.tf
prod	- /prod/base-infra/terraformstate.tf - /prod/infra-microservices/terraformstate.tf - /prod/frontend-infra/terraformstate.tf - /prod/contact-center/terraformstate.tf

Microservices architecture

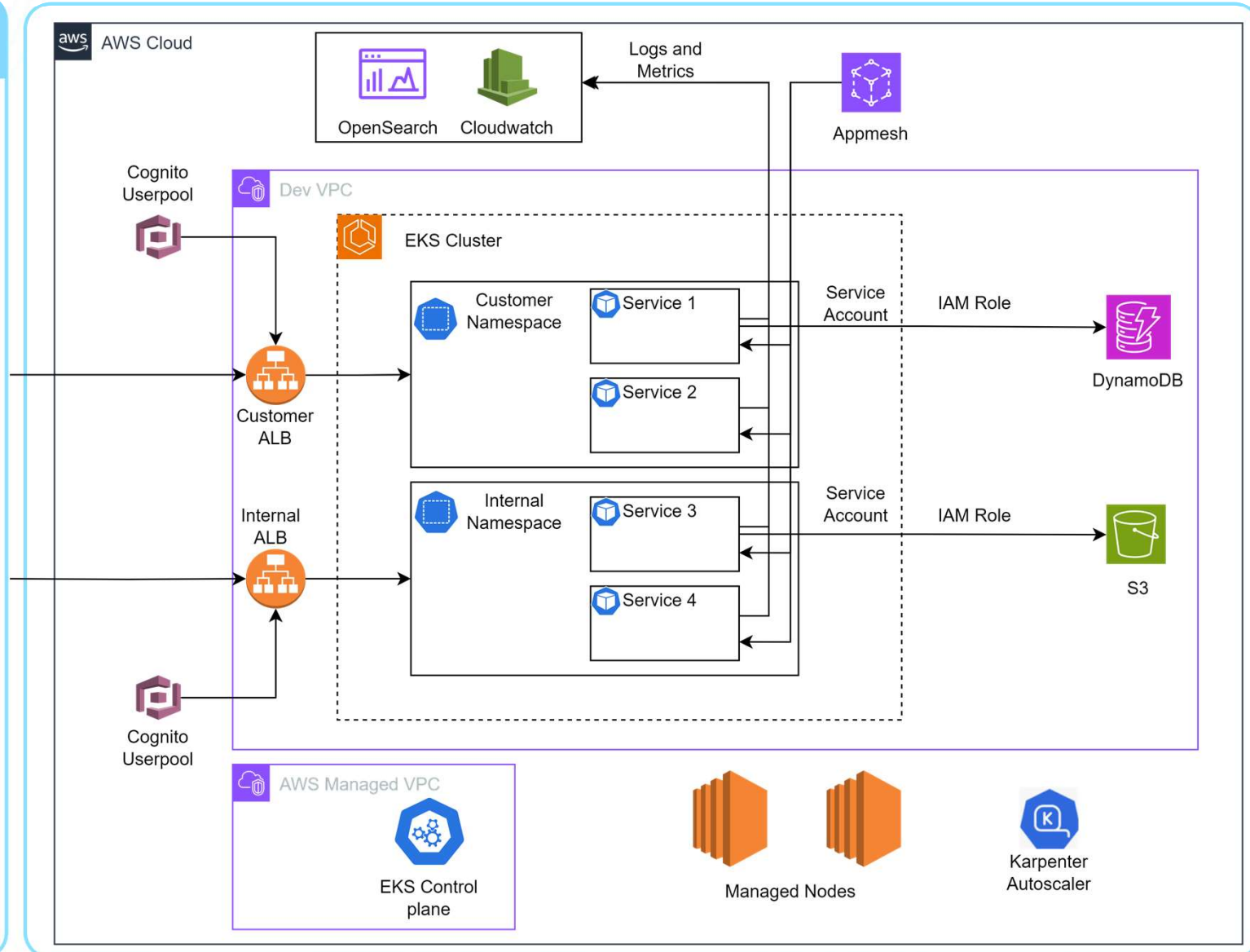
www.trianz.com



Kubernetes Internals



- This slide illustrates the internal structure of each EKS cluster, with only one shown for clarity.
- Each cluster will have two namespaces: customer and internal.
- For customers:
 - A dedicated ALB, Cognito user pool, and namespace will be deployed to service the APIs.
- For internal purposes:
 - A separate ALB, Cognito (with SAML integration with Azure AD), and internal namespace will be deployed.
- Service roles for each service will be mapped to IAM roles to provide access to AWS services like DynamoDB and S3 bucket.
- All logging from the microservices will be routed to CloudWatch. Fluent-bit will be running in every pod to forward the logs. AWS OpenSearch Service will be leveraged for observability.
- The services of the EKS cluster will run on managed nodes.
- Karpenter auto-scaler must be leveraged for dynamic provisioning of new nodes of need basis.
- App Mesh provides application-level networking to make it easy for your services to communicate with each other
- EKS cluster itself will run on a fully managed VPC to enable communication with the on-prem or internet



Thank you

yaseen.mohammed@trianz.com



The content in this document is copyrighted; any unauthorized use – in part or full – may violate the copyright, trademark, and other laws. This document may not be modified, reproduced or publicly displayed, performed or distributed, or used for any public or commercial purposes. The Trianz name and its products are subject to trademark and copyright protections, regardless of how and where referenced.